

Autonomous Testing: AI Agents Taking Over Quality Assurance

By [Donato Giuliano - UO318987], [Marvin Kickel - UO318535], [Daniel Onís Fabián – UO293946] — *Software Architecture, 2025-2026*

This document presents what autonomous testing is, how AI agents work under the hood, and its practical implications for software teams, based on a conversation with Vilhelm von Ehrenheim, co-founder and Chief AI Officer of QA.Tech, on Software Engineering Radio Episode 674.

The Evolution & Fundamentals of Autonomous Testing

Testing has evolved through three phases. Manual testing was the natural starting point — developers interact with the application by hand to verify it works.

Automated testing followed, with tools like Selenium and Cypress allowing teams to run hardcoded scripts. However, these scripts are fragile — any UI change breaks them, forcing engineers to spend more time maintaining tests than improving quality. Autonomous testing raises the abstraction level further: instead of scripting exact steps, you define high-level objectives and an AI agent figures out how to achieve them, adapting in real time.

The key innovation is self-healing. When a button moves or a new step is added to a checkout flow, the agent understands the objective and adapts without failing. Before adopting autonomous testing, however, companies need a dedicated staging environment — running tests directly on production is not viable. Von Ehrenheim shares a real example: a company that had to downsize its QA team during a financial crisis struggled to maintain their test suite. After partnering with QA.Tech, the remaining team could focus on strategic work rather than repetitive browser tasks, reducing testing costs drastically.

Under the Hood: AI Agents in Action

An autonomous testing agent is a system of five components working in a continuous loop. The **Observer** reads the browser and captures the current state of the page. **Memory** combines that with context from previous steps in the session. The **Core Model** — the AI brain — decides what to do next based on both. The

Planner translates that decision into specific ordered steps. Finally, the **Executor** carries them out in the browser. After each action, the cycle restarts.

This architecture allows agents to recover from unexpected situations. In one case, a client replaced their login system with a third-party OAuth2 provider. Traditional tests broke immediately. The agent read the new screen, identified the login fields, and completed authentication without any manual updates. In another case, a database reset left no warehouse data available. Rather than failing, the agent created a new warehouse from scratch, configured it, and confirmed the feature was working. These examples illustrate the core advantage: agents understand objectives, not just steps.

Implementation, Metrics and the Evolving Role of QA

Measuring the success of autonomous testing comes down to three key metrics. Test execution time captures how much faster testing cycles become when manual bottlenecks are removed. The ratio of flaky tests — tests that fail intermittently for no real reason — typically drops significantly, as agents use visual cues like loading spinners rather than hardcoded wait times. Finally, the bug detection rate is what Von Ehrenheim calls "the holy grail" — tracking how many real bugs the system actually finds and prevents.

Adoption is not without hurdles. Technically, complex authentication flows and sensitive data in staging environments can block implementation entirely — some clients had to abandon the tool because they could not share regulated data with an external provider. Organisationally, QA teams sometimes hesitate out of fear of being replaced. Von Ehrenheim reframes this as gaining superpowers — testers transition from maintaining broken scripts to acting as QA Managers who design high-level test strategies and work closely with development teams. We are currently in a semi-autonomous phase: AI handles the repetitive work, but human oversight remains essential for defining goals and understanding context.

References

von Ehrenheim, V. (2024). Autonomous Testing. Software Engineering Radio, Episode 674. IEEE Computer Society. <https://se-radio.net/2024/episode-674>

QA.Tech — <https://qa.tech>

Applitoools — <https://applitoools.com>

Meticulous — <https://meticulous.ai>